CPE 486/586: Machine Learning for Engineers

10 Dimensionality Reduction

Fall 2026

Rahul Bhadani

Outline

1. Motivation
2. Principal Component Analysis (PCA)
3. Mathematics behind PCA
4. PCA by Hand
5. Limitations and Beyond PCA

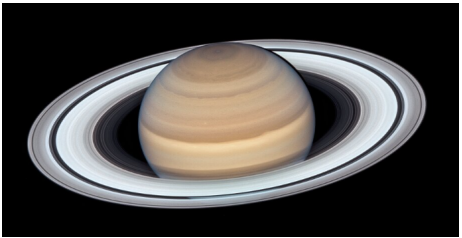


Motivation

Examples of High-Dimensional Data

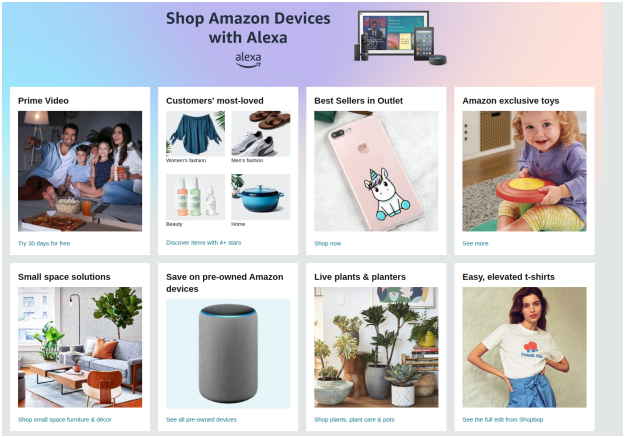
- High resolution images





Examples of High-Dimensional Data (cont.)

- Customer Purchase Data



Motivating Dimensionality Reduction

- There is a curse of dimensionality! The complexity of a model increases with the dimensionality. Sometimes exponentially!
- So, why should we perform dimensionality reduction?
  - reduces the time complexity: less computation
  - reduces the space complexity: fewer parameters
  - saves costs: some features/variables cost money
  - makes complex high-dimensional data easier to interpret
  - Can you visualize data with more than three dimensions?

The Curse of Dimensionality in Numbers

Spread points uniformly in the unit hypercube $[0, 1]^D$. A neighborhood that has 10% of the data is a sub-cube with edge length

$$e_D = 0.1^{1/D} \tag{1}$$

D	1	2	3	10	100
e_D	0.100	0.316	0.464	0.794	0.977

Consequence

At $D = 10$ the "local" neighborhood covers 79% of the range of a feature. At $D = 100$ it covers 98%. A k-NN classifier then averages over points that lie far from the query.

Learning Representations

Dimensionality Reduction Algorithms

Powerful, often unsupervised, learning techniques for extracting hidden and potentially lower dimensional structure from high dimensional datasets.

Examples:
PCA, Kernel PCA, ICA, CCA, t-SNE, UMAP, Autoencoders, Matrix Factorization

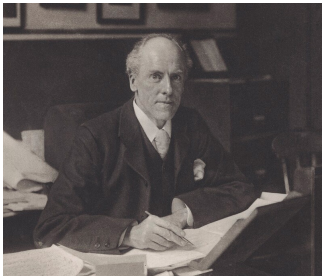
Some advantage:

- Better Visualization
- Efficient use of resources such as time, memory, communication
- Statistical: fewer parameters to fit, which reduces overfitting when the discarded directions carry noise
- Noise removal (improving data quality)



Principal Component Analysis (PCA)

PCA: Proposed more than 100 years ago!



Karl Pearson (invented in 1901)



Harold Hotelling (named it principal components, 1933)

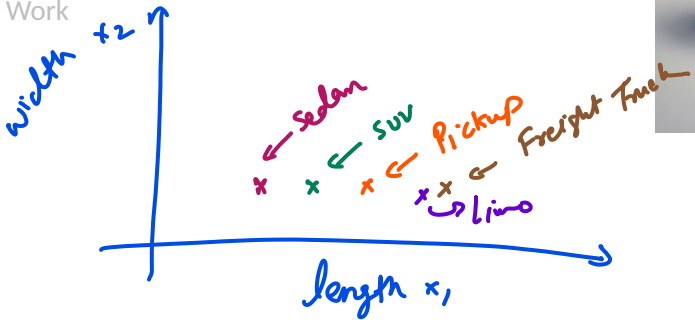
Why?

- Helpful for reducing number of features
- For 2D/3D visualization
- Helpful in unsupervised algorithms
- Data storage cost can be lowered

Intuition Behind PCA

x_1: length of the car, x_2: width of the car

Class Work

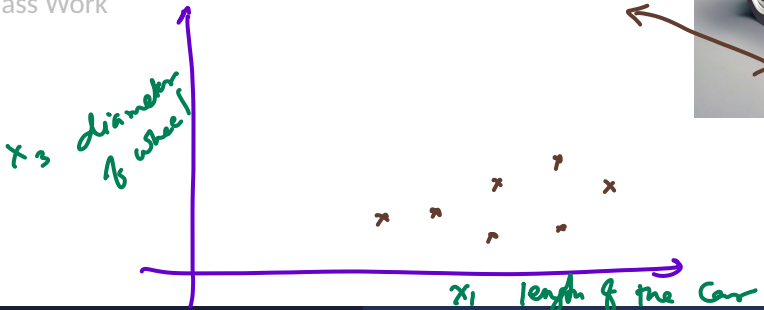


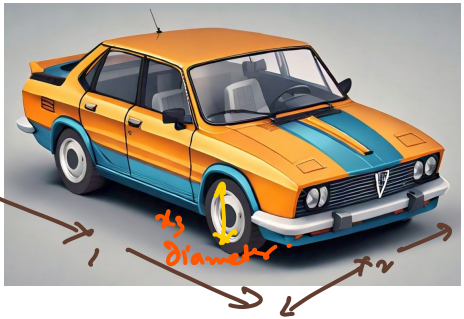


Intuition Behind PCA

x_1: length of the car, x_3: diameter of the wheel

Class Work



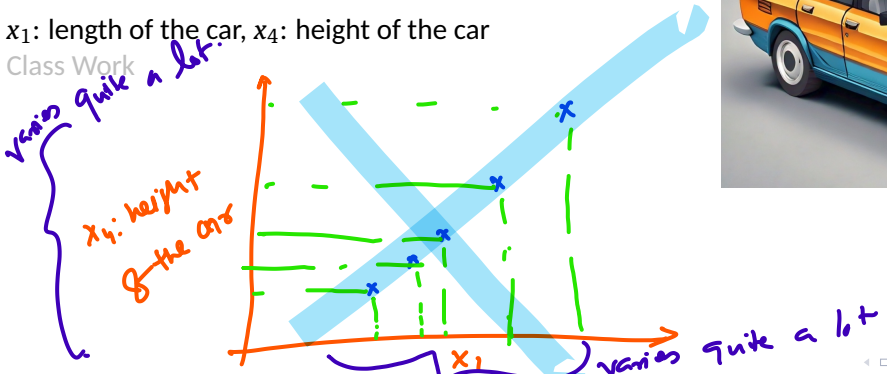


Intuition Behind PCA

x_1: length of the car, x_4: height of the car

Class Work





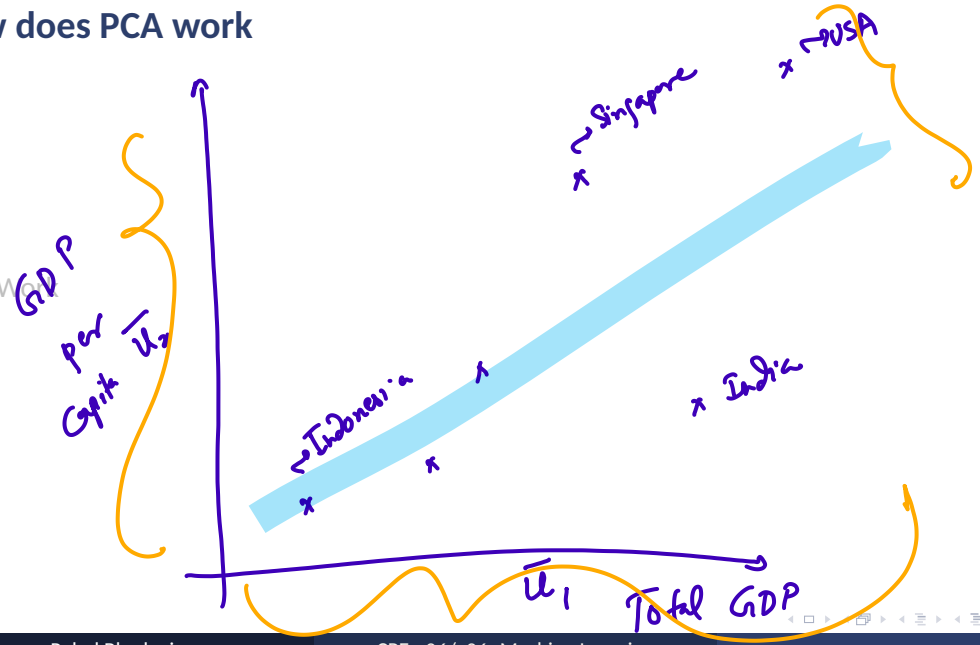
Problem Setting

We are interested in finding projections $\hat{\mathbf{x}}_i$ of data points $\mathbf{x}_i$ that are as similar to the original data points as possible, but which have significantly lower intrinsic dimensionality.

Question

Given a dataset $X = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$, $\mathbf{x}_i \in \mathbb{R}^D$, is there an M-dimensional subspace of $\mathbb{R}^D$, $M \ll D$, in which X approximately lies?

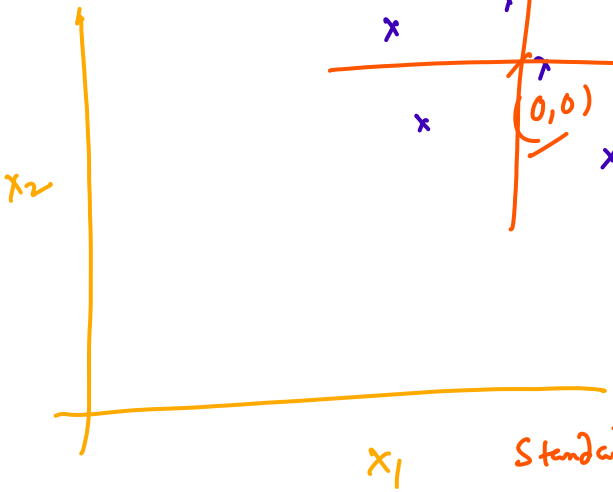
How does PCA work



Class Work

How does PCA work

Class Work



min-max Scaling

$$\frac{x - x_{min}}{x_{max} - x_{min}}$$

Standard Scaling

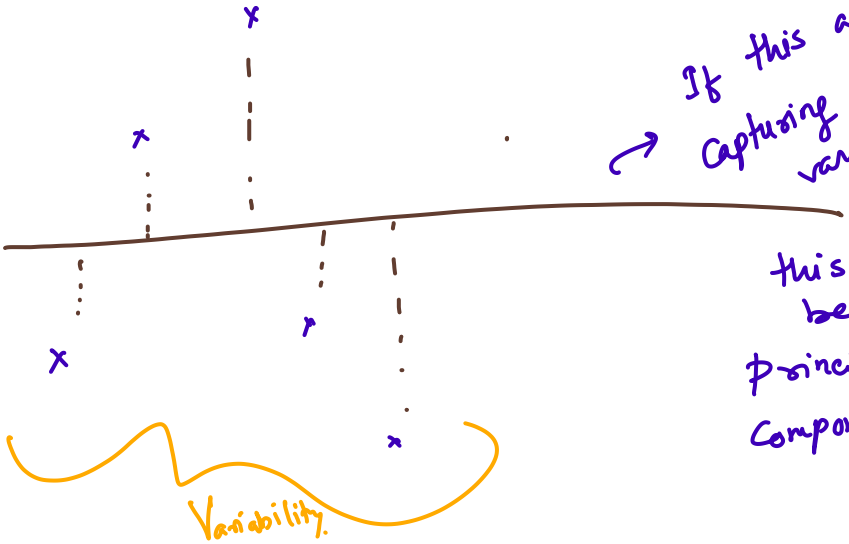
$$z = \frac{x - \mu}{\sigma^2}$$

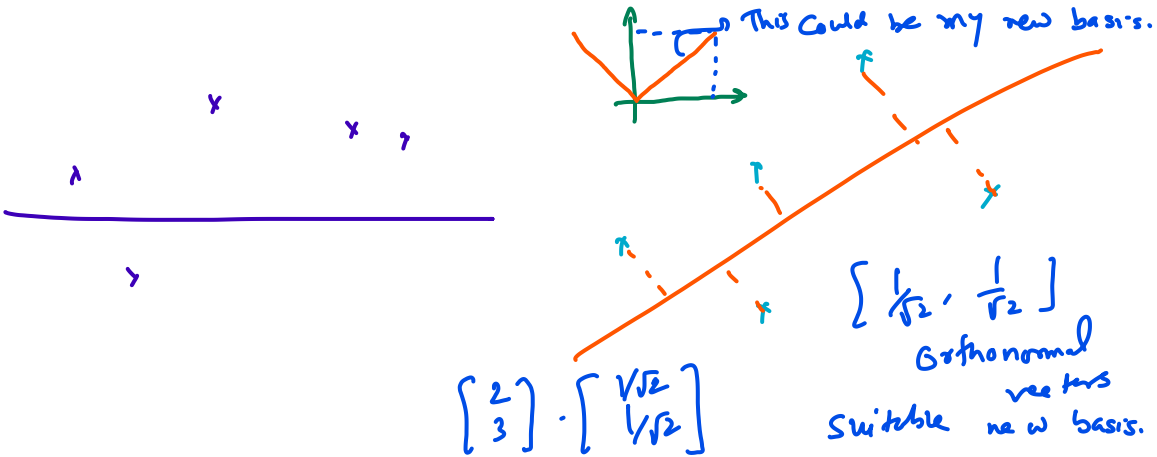
$\mu \rightarrow$ mean.

$\sigma^2 \rightarrow$ Variance

How does PCA work

Class Work





% Mathematics behind PCA

Feature Preprocessing

- Center the data so that the features have mean 0.

$$\boldsymbol{\mu} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{x}_i, \qquad \mathbf{x}_i \leftarrow \mathbf{x}_i - \boldsymbol{\mu} \tag{2}$$

- When the features carry different units, also scale them to $\sigma_j = 1$.

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N} x_{ij}^2, \qquad x_{ij} \leftarrow \frac{x_{ij}}{\sigma_j} \tag{3}$$

After standardization, a speed recorded in km/h and again in mph gives two identical columns, and PCA merges them into one component.

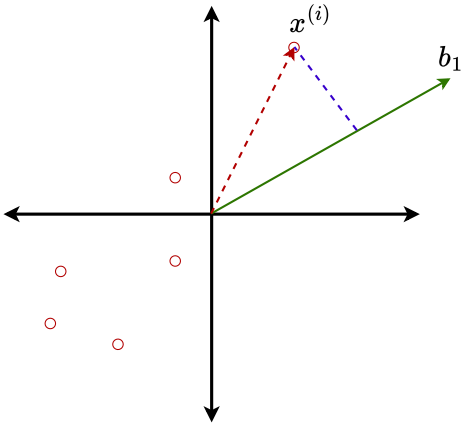
Covariance Matrix

We assume the samples $X = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$ are iid. After centering, they have mean $\mathbf{0}$ and covariance matrix

$$\Sigma = \frac{1}{N} \sum_{i=1}^{N} \mathbf{x}_i \mathbf{x}_i^\top \in \mathbb{R}^{D \times D} \tag{4}$$

Here $\mathbf{x}_i \in \mathbb{R}^D$.

Projecting onto New Axes



The coordinates (the code) of $\mathbf{x}_i$ along the new axes $\mathbf{b}_1, \ldots, \mathbf{b}_M$ are

$$\mathbf{z}_i = \mathbf{B}^\top \mathbf{x}_i \in \mathbb{R}^M$$

with $\mathbf{B} := [\mathbf{b}_1, \ldots, \mathbf{b}_M] \in \mathbb{R}^{D \times M}$. The projection of $\mathbf{x}_i$ in $\mathbb{R}^D$ is $\mathbf{B}\mathbf{B}^\top \mathbf{x}_i$.
We assume that the columns of $\mathbf{B}$ are orthonormal: $\|\mathbf{b}_k\| = 1$ and $\mathbf{b}_k^\top \mathbf{b}_l = 0$ for $k \neq l$.

So standard basis
(Refresh your Linear Algebra)

Find the Direction of Maximal Variance

We maximize the variance of the low-dimensional code using a sequential approach. Start with a single vector $\mathbf{b}_1 \in \mathbb{R}^D$ that maximizes the variance of the projected data.

How do we choose unit vector $\mathbf{b}_1$ so that we maximize

$$V = \frac{1}{N} \sum_{i=1}^{N} (\mathbf{b}_1^\top \mathbf{x}_i)^2$$

The data are centered, so the projections $\mathbf{b}_1^\top \mathbf{x}_i$ have mean 0 and their mean square is their variance.

The Direction with Maximal Variance

Expanding

$$V = \frac{1}{N}\sum_{i=1}^{N}(\mathbf{b}_1^\top \mathbf{x}_i)^2 = \frac{1}{N}\sum_{i=1}^{N}\mathbf{b}_1^\top \mathbf{x}_i \mathbf{x}_i^\top \mathbf{b}_1$$
$$= \mathbf{b}_1^\top \left(\frac{1}{N}\sum_{i=1}^{N}\mathbf{x}_i \mathbf{x}_i{}^\top\right)\mathbf{b}_1 = \mathbf{b}_1^\top \Sigma \mathbf{b}_1 \quad (5)$$

The dot product is symmetric, i.e. $\mathbf{b}_1^\top \mathbf{x}_i = \mathbf{x}_i^\top \mathbf{b}_1$.

Lagrange Multiplier to Solve the Optimization Problem

Our goal is to maximize $\mathbf{b}_1^\top \Sigma \mathbf{b}_1$ such that

$$\mathbf{b}_1^\top \mathbf{b}_1 = 1 \Rightarrow \mathbf{b}_1^\top \mathbf{b}_1 - 1 = 0$$

We use the method of Lagrange Multiplier

$$\mathcal{L}(\mathbf{b}_1, \lambda) = \mathbf{b}_1^\top \Sigma \mathbf{b}_1 - \lambda(\mathbf{b}_1^\top \mathbf{b}_1 - 1)$$

Lagrange Multiplier to Solve the Optimization Problem

Essentially this is square.

$$\mathcal{L}(\mathbf{b}_1, \lambda) = \mathbf{b}_1^\top \Sigma \mathbf{b}_1 - \lambda(\mathbf{b}_1^\top \mathbf{b}_1 - 1) \quad (6)$$

Take the gradient with respect to $\mathbf{b}_1$

$$\nabla_{\mathbf{b}_1} \mathcal{L}(\mathbf{b}_1, \lambda) = 2\Sigma\mathbf{b}_1 - 2\lambda\mathbf{b}_1 = \mathbf{0}$$
$$\Rightarrow \Sigma\mathbf{b}_1 = \lambda\mathbf{b}_1 \quad (7)$$

We see that $\mathbf{b}_1$ is an eigenvector of the covariance matrix Σ and the Lagrange multiplier λ is the corresponding eigenvalue.

Eigenvalue Problem

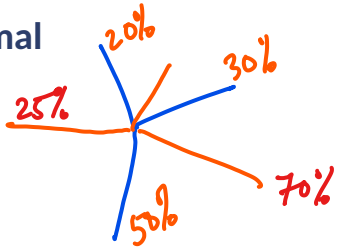
From the eigenvector property:

$$V = \mathbf{b}_1^\top \Sigma \mathbf{b}_1 = \lambda \mathbf{b}_1^\top \mathbf{b}_1 = \lambda \tag{8}$$

Hence, the variance of the data projected onto a 1-D subspace equals the eigenvalue associated with the basis vector $\mathbf{b}_1$ that spans the subspace.

To maximize the variance of the low-dimensional code, we choose the eigenvector associated with the largest eigenvalue λ_1 of the covariance matrix. This eigenvector is called the **first principal component**.

Why the Principal Directions Are Orthonormal



Σ is symmetric, since $\Sigma^\top = \frac{1}{N}\sum_i (\mathbf{x}_i \mathbf{x}_i^\top)^\top = \Sigma$.
For a vector $\mathbf{w} \in \mathbb{R}^D$,

$$\mathbf{w}^\top \Sigma \mathbf{w} = \frac{1}{N}\sum_{i=1}^{N} (\mathbf{w}^\top \mathbf{x}_i)^2 \geq 0 \tag{9}$$

so Σ is positive semidefinite.

Spectral Theorem

A real symmetric matrix factors as $\Sigma = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$ with $\mathbf{U} = [\mathbf{b}_1, \ldots, \mathbf{b}_D]$, $\mathbf{U}^\top\mathbf{U} = \mathbf{I}$ and $\mathbf{\Lambda} = \mathrm{diag}(\lambda_1, \ldots, \lambda_D)$, where $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_D \geq 0$.

$\mathbf{B}$ keeps the first M columns of $\mathbf{U}$, so its columns are orthonormal, as we assumed.

The Second Principal Component

Maximize $\mathbf{b}_2^\top \Sigma \mathbf{b}_2$ subject to $\mathbf{b}_2^\top \mathbf{b}_2 = 1$ and $\mathbf{b}_2^\top \mathbf{b}_1 = 0$.

$$\begin{aligned} \mathcal{L}(\mathbf{b}_2, \lambda, \phi) &= \mathbf{b}_2^\top \Sigma \mathbf{b}_2 - \lambda(\mathbf{b}_2^\top \mathbf{b}_2 - 1) - \phi\, \mathbf{b}_2^\top \mathbf{b}_1 \\ \nabla_{\mathbf{b}_2} \mathcal{L} &= 2\Sigma \mathbf{b}_2 - 2\lambda \mathbf{b}_2 - \phi\, \mathbf{b}_1 = \mathbf{0} \end{aligned} \tag{10}$$

Multiply on the left by $\mathbf{b}_1^\top$. Since $\mathbf{b}_1^\top \Sigma \mathbf{b}_2 = (\Sigma \mathbf{b}_1)^\top \mathbf{b}_2 = \lambda_1 \mathbf{b}_1^\top \mathbf{b}_2 = 0$,

$$0 - 0 - \phi = 0 \;\Rightarrow\; \phi = 0 \;\Rightarrow\; \Sigma \mathbf{b}_2 = \lambda \mathbf{b}_2 \tag{11}$$

$\mathbf{b}_2$ is the eigenvector with the second largest eigenvalue λ_2. Repeating the argument gives $\mathbf{b}_3, \ldots, \mathbf{b}_M$.

Principal Components

The eigenvalue equation has D solutions, the eigenpairs $(\lambda_j, \mathbf{b}_j)$ for $j = 1, \ldots, D$.
To reduce data in $\mathbb{R}^6$ to $\mathbb{R}^2$, keep $\mathbf{b}_1$ and $\mathbf{b}_2$, the eigenvectors with the two largest eigenvalues.

Note

$\mathbf{b}_1, \mathbf{b}_2, \ldots$ are the new basis vectors for the reduced data.

Reduced Dimensions

The reduced representation (code) $\mathbf{z}_i$ is

$$\mathbf{z}_i = \begin{bmatrix} \mathbf{b}_1^\top \mathbf{x}_i & \leftarrow \text{projection onto } \mathbf{b}_1 \\ \mathbf{b}_2^\top \mathbf{x}_i & \leftarrow \text{projection onto } \mathbf{b}_2 \\ \vdots & \vdots \\ \mathbf{b}_M^\top \mathbf{x}_i & \leftarrow \text{projection onto } \mathbf{b}_M \end{bmatrix} \tag{12}$$

M is the new reduced dimension.

Here, $\mathbf{z}_i = \mathbf{B}^\top \mathbf{x}_i \in \mathbb{R}^M$.

Reconstruction

We reconstruct the centered point $\mathbf{x}_i$ from its code, with some loss of information:

$$\hat{\mathbf{x}}_i = \mathbf{B}\mathbf{z}_i = \sum_{j=1}^{M} z_{ij}\mathbf{b}_j \tag{13}$$

For raw data, add the mean back: $\hat{\mathbf{x}}_i = \boldsymbol{\mu} + \mathbf{B}\mathbf{z}_i$.

Note

The quality of the reconstruction depends on the number of principal components M. With $M = D$ the reconstruction is exact. With $M < D$ it loses information unless the discarded eigenvalues are zero.

Reconstruction Error

$$\text{Error} = \frac{1}{N}\sum_{i=1}^{N} \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 \tag{14}$$

This error measures how well the reduced-dimensional data approximate the original data. The next two frames show that PCA minimizes it.

Note

The reconstruction error equals the sum of the eigenvalues of the discarded components.

Maximum Variance Equals Minimum Error (1/2)

Take an orthonormal basis $\mathbf{b}_1, \ldots, \mathbf{b}_D$ of $\mathbb{R}^D$ whose first M vectors span the kept subspace. A centered point is

$$\mathbf{x}_i = \sum_{j=1}^{D} z_{ij}\mathbf{b}_j, \qquad z_{ij} = \mathbf{b}_j^\top \mathbf{x}_i \tag{15}$$

Keeping the first M terms leaves the residual

$$\mathbf{x}_i - \hat{\mathbf{x}}_i = \sum_{j=M+1}^{D} z_{ij}\mathbf{b}_j \tag{16}$$

Orthonormality removes the cross terms:

$$\|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 = \sum_{j=M+1}^{D} z_{ij}^2 \tag{17}$$

Maximum Variance Equals Minimum Error (2/2)

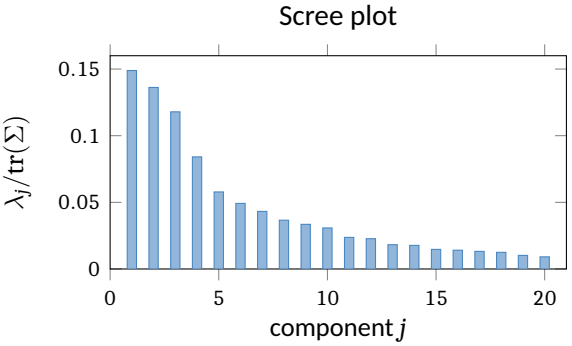
Average over the N points. The variance along $\mathbf{b}_j$ is $\frac{1}{N}\sum_i z_{ij}^2 = \mathbf{b}_j^\top \Sigma \mathbf{b}_j$, so

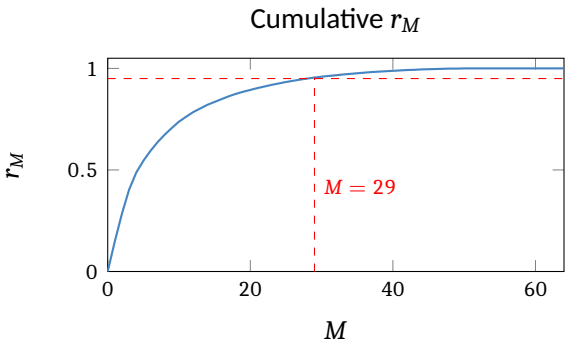
$$\text{Error} = \sum_{j=M+1}^{D} \mathbf{b}_j^\top \Sigma \mathbf{b}_j = \text{tr}(\Sigma) - \underbrace{\sum_{j=1}^{M} \mathbf{b}_j^\top \Sigma \mathbf{b}_j}_{\text{retained variance}} \tag{18}$$

In an orthonormal basis $\sum_{j=1}^{D} \mathbf{b}_j^\top \Sigma \mathbf{b}_j = \text{tr}(\Sigma)$, which the data fix. Minimizing the error is maximizing the retained variance. With the eigenvectors, $\mathbf{b}_j^\top \Sigma \mathbf{b}_j = \lambda_j$ and Error $= \sum_{j=M+1}^{D} \lambda_j$.

Pearson's closest-fit subspace (1901) and Hotelling's largest-variance subspace (1933) are the same subspace.

Choosing M: Explained Variance on Real Data





Handwritten digits, `sklearn.datasets.load_digits`: $N = 1797$ images of 8×8 pixels, so $D = 64$. The pixels are centered and left unscaled.

$$r_M = \frac{\sum_{j=1}^{M} \lambda_j}{\sum_{j=1}^{D} \lambda_j}, \qquad r_{13} = 0.803, \quad r_{21} = 0.903, \quad r_{29} = 0.955, \quad r_{41} = 0.990 \tag{19}$$

The scree plot shows no sharp elbow for these data. A variance threshold gives a definite M.

Using Singular Value Decomposition for PCA

After centering the dataset, the covariance matrix is $\Sigma = \frac{1}{N}\mathbf{X}\mathbf{X}^\top$ in matrix representation.

Let $\mathbf{U}$ be the matrix of eigenvectors, sorted by eigenvalue with the largest first. The PCA transformation of the data is $\mathbf{Z} = \mathbf{U}^\top\mathbf{X}$. Keeping the first M rows of $\mathbf{Z}$ projects the data onto M dimensions.

When using the data matrix $\mathbf{X}$ notation, we follow a convention where the rows denote the features from 1 to D and the columns are the samples from 1 to N.

Using Singular Value Decomposition for PCA

With $\mathbf{X} \in \mathbb{R}^{D \times N}$ centered, take $\mathbf{X} = \mathbf{Q}\mathbf{\Gamma}\mathbf{V}^\top$ with $\mathbf{Q} \in \mathbb{R}^{D \times D}$, $\mathbf{\Gamma} \in \mathbb{R}^{D \times N}$, $\mathbf{V} \in \mathbb{R}^{N \times N}$.

$$\Sigma = \frac{1}{N}\mathbf{X}\mathbf{X}^\top = \mathbf{Q}\left(\frac{1}{N}\mathbf{\Gamma}\mathbf{\Gamma}^\top\right)\mathbf{Q}^\top \;\Rightarrow\; \mathbf{U} = \mathbf{Q}, \quad \lambda_j = \frac{\gamma_j^2}{N} \tag{20}$$

$$\mathbf{Z} = \mathbf{U}^\top\mathbf{X} = \mathbf{Q}^\top\mathbf{Q}\,\mathbf{\Gamma}\mathbf{V}^\top = \mathbf{\Gamma}\mathbf{V}^\top$$

Row j of $\mathbf{Z}$ is $\gamma_j \mathbf{v}_j^\top$. Keep the first M rows.

Why SVD in practice

SVD never forms $\mathbf{X}\mathbf{X}^\top$, whose condition number is the square of the condition number of $\mathbf{X}$. The small eigenvalues keep more of their correct digits.

PCA as an Algorithm

Fit on the training data $\mathbf{X}_{\text{train}} \in \mathbb{R}^{D \times N}$:

1. Compute $\boldsymbol{\mu}$. Set $\mathbf{S} = \operatorname{diag}(\sigma_1, \ldots, \sigma_D)$ when the features carry different units, and $\mathbf{S} = \mathbf{I}$ otherwise.
2. Standardize: $\mathbf{X} \leftarrow \mathbf{S}^{-1}(\mathbf{X} - \boldsymbol{\mu}\mathbf{1}^\top)$.
3. SVD $\mathbf{X} = \mathbf{Q}\boldsymbol{\Gamma}\mathbf{V}^\top$, so $\lambda_j = \gamma_j^2 / N$.
4. Choose M from r_M. $\mathbf{B}$ holds the first M columns of $\mathbf{Q}$.

Transform a new point $\mathbf{x}$:

$$\mathbf{z} = \mathbf{B}^\top \mathbf{S}^{-1}(\mathbf{x} - \boldsymbol{\mu}), \qquad \hat{\mathbf{x}} = \boldsymbol{\mu} + \mathbf{S}\mathbf{B}\mathbf{z} \tag{21}$$

$\boldsymbol{\mu}$, $\mathbf{S}$ and $\mathbf{B}$ come from the training set. Refitting them on test data leaks information into the evaluation.



PCA by Hand

PCA by Hand: Data and Centering

Four samples with two features in the same units. We center and skip the rescaling.

i	x_{i1}	x_{i2}	$x_{i1} - \mu_1$	$x_{i2} - \mu_2$
1	-13	-2	-15	-5
2	1	-4	-1	-7
3	9	2	7	-1
4	11	16	9	13
mean	2	3	0	0

$$\boldsymbol{\mu} = \frac{1}{4}\begin{bmatrix} -13 + 1 + 9 + 11 \\ -2 - 4 + 2 + 16 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \end{bmatrix} \tag{22}$$

PCA by Hand: Covariance Matrix

$$\Sigma_{11} = \tfrac{1}{4}\left[(-15)^2 + (-1)^2 + 7^2 + 9^2\right] = \tfrac{356}{4} = 89$$
$$\Sigma_{12} = \tfrac{1}{4}\left[(-15)(-5) + (-1)(-7) + (7)(-1) + (9)(13)\right] = \tfrac{192}{4} = 48 \tag{23}$$
$$\Sigma_{22} = \tfrac{1}{4}\left[(-5)^2 + (-7)^2 + (-1)^2 + 13^2\right] = \tfrac{244}{4} = 61$$

$$\Sigma = \begin{bmatrix} 89 & 48 \\ 48 & 61 \end{bmatrix}, \qquad \mathrm{tr}(\Sigma) = 150, \qquad \det(\Sigma) = 3125 \tag{24}$$

The positive off-diagonal entry says the two features rise together. Their correlation is $48/\sqrt{89 \cdot 61} = 0.65$.

PCA by Hand: Eigenvalues and Eigenvectors

$$\det(\Sigma - \lambda \mathbf{I}) = (89-\lambda)(61-\lambda) - 48^2 = \lambda^2 - 150\lambda + 3125 = 0$$
$$\Rightarrow \; (\lambda - 125)(\lambda - 25) = 0 \; \Rightarrow \; \lambda_1 = 125, \;\; \lambda_2 = 25 \tag{25}$$

$\lambda_1 = 125$:

$$\begin{bmatrix} -36 & 48 \\ 48 & -64 \end{bmatrix} \mathbf{b}_1 = \mathbf{0} \; \Rightarrow \; \mathbf{b}_1 = \begin{bmatrix} 0.8 \\ 0.6 \end{bmatrix} \tag{26}$$

$\lambda_2 = 25$:

$$\begin{bmatrix} 64 & 48 \\ 48 & 36 \end{bmatrix} \mathbf{b}_2 = \mathbf{0} \; \Rightarrow \; \mathbf{b}_2 = \begin{bmatrix} -0.6 \\ 0.8 \end{bmatrix} \tag{27}$$

Checks: $\lambda_1 + \lambda_2 = 150 = \mathrm{tr}(\Sigma)$, $\lambda_1 \lambda_2 = 3125 = \det(\Sigma)$, $\mathbf{b}_1^\top \mathbf{b}_2 = -0.48 + 0.48 = 0$.
PC1 explains $125/150 = 83.3\%$ of the variance.

PCA by Hand: Project onto PC1 and Reconstruct

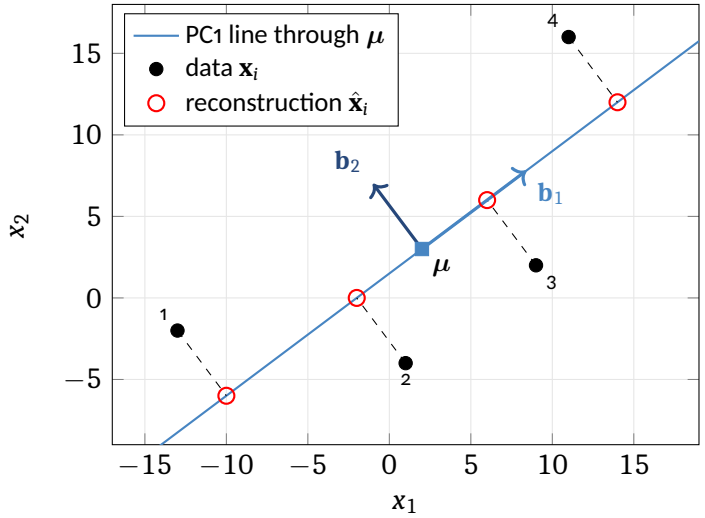
$$z_i = \mathbf{b}_1^\top(\mathbf{x}_i - \boldsymbol{\mu}), \qquad \hat{\mathbf{x}}_i = \boldsymbol{\mu} + z_i\,\mathbf{b}_1$$

i	z_i	$\hat{\mathbf{x}}_i$	$\mathbf{x}_i - \hat{\mathbf{x}}_i$	$\lVert\mathbf{x}_i - \hat{\mathbf{x}}_i\rVert^2$
1	$-12 - 3 = -15$	$(-10, -6)$	$(-3, 4)$	25
2	$-0.8 - 4.2 = -5$	$(-2, 0)$	$(3, -4)$	25
3	$5.6 - 0.6 = 5$	$(6, 6)$	$(3, -4)$	25
4	$7.2 + 7.8 = 15$	$(14, 12)$	$(-3, 4)$	25

$$\tfrac{1}{4}\textstyle\sum_i z_i^2 = \tfrac{500}{4} = 125 = \lambda_1, \qquad \text{Error} = \tfrac{100}{4} = 25 = \lambda_2 \tag{28}$$

The residuals point along $\pm\mathbf{b}_2$. The 1-D code discards the variance along PC2 and nothing more.

PCA by Hand: The Picture



PCA by Hand: Checking with Python

```python
import numpy as np
from sklearn.decomposition import PCA

X = np.array([[-13, -2], [1, -4], [9, 2], [11, 16]])  # N x D
Xc = X - X.mean(axis=0)
Sigma = Xc.T @ Xc / len(X)            # 1/N, as on the slides
lam, B = np.linalg.eigh(Sigma)        # ascending order
print(lam)                            # [ 25. 125.]

pca = PCA().fit(X)
print(pca.explained_variance_)        # [166.67  33.33], uses 1/(N-1)
print(pca.explained_variance_ratio_)  # [0.833 0.167], same ratio
print(pca.components_)                # rows are b1, b2, sign may flip
print(pca.transform(X)[:, 0])         # z_i = [-15. -5.  5. 15.]
```

Conventions to watch. `sklearn` stores samples as rows, the transpose of our $\mathbf{X}$, and divides by $N - 1$. An eigenvector is fixed up to sign, so $-\mathbf{b}_1$ is an equally valid answer.



Limitations and Beyond PCA

To Standardize or Not

Rescale x_1 of the worked example by 10, say centimeters to millimeters:

$$\Sigma' = \begin{bmatrix} 8900 & 480 \\ 480 & 61 \end{bmatrix} \Rightarrow \lambda'_1 = 8926.0, \quad r_1 = 0.996 \tag{29}$$

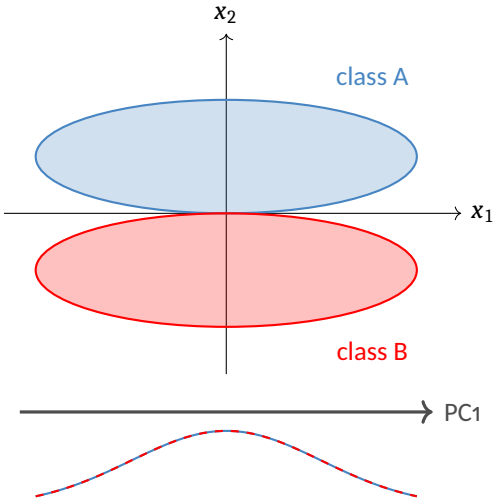
PC1 now lies 3.1° off the x_1 axis and explains 99.6% of the variance, up from 83.3%. The points are the same. A change of unit moved PC1.

Rule of thumb

Standardize when the features carry different units, such as meters and kilograms. Keep the raw scale when the units match and the spread itself carries meaning, as with pixel intensities.

A pixel that stays blank across the data set has $\sigma_j = 0$, and x_{ij}/σ_j is undefined. The digits data contain three such pixels. `StandardScaler` leaves those columns unscaled.

PCA Ignores the Labels



z_1 has the same density in both classes

Two classes with equal priors. Within a class, $\sigma = 3$ along x_1 and $\sigma = 0.3$ along x_2. The class means sit at $x_2 = \pm 0.6$.

$$\mathrm{Var}(x_1) = 3^2 = 9$$

$$\mathrm{Var}(x_2) = \underbrace{0.3^2}_{\text{within}} + \underbrace{0.6^2}_{\text{between}} = 0.45 \tag{30}$$

PC1 is the x_1 axis and keeps $9/9.45 = 95\%$ of the variance. A classifier that sees z_1 alone is right half the time.

PCA ranks directions by spread. Here the class information sits in the low-variance direction. LDA uses the labels and picks x_2.

Beyond PCA

Method	Map	What it preserves	Embeds a new point?
PCA	linear	directions of largest variance	yes, $\mathbf{B}^\top(\mathbf{x} - \boldsymbol{\mu})$
Kernel PCA	nonlinear	variance in a kernel feature space	yes, via kernel values with the training points
ICA	linear	independent, non-Gaussian sources	yes, the unmixing matrix
t-SNE	nonlinear	local neighborhoods, for 2-D or 3-D plots	no direct map
UMAP	nonlinear	local neighborhoods and part of the global layout	yes, approximate
Autoencoder	nonlinear	reconstruction through a bottleneck	yes, the encoder

A link back to PCA

A linear autoencoder trained with squared error spans the subspace of the top M principal components (Baldi and Hornik, 1989). Nonlinear activations let it learn a curved manifold.

Cluster sizes and gaps in a t-SNE plot carry little meaning. Read the neighborhoods.

Python Notebook

https://github.com/rahulbhadani/CPE486586_FA26/blob/master/Code/CPE486586_Ch10_Dimensionality_Reduction.ipynb

The End